

ESM 5734
INTRODUCTION TO THE FINITE ELEMENT METHOD
FINAL PROJECT

**APPROXIMATE SOLUTION OF 2D POISSON EQUATION USING
FINITE ELEMENT METHOD IN AN ELLIPTIC DOMAIN**

RUHIT SINHA
SAADBIN KHAN

Department of Biomedical Engineering & Mechanics
Virginia Tech

December 14, 2022

Table of Contents

1 Problem Statement	1
2 Methodology	2
2.1 Weak formulation	2
2.2 Matrix formulation by Galerkin approximation	3
2.3 Meshing	4
2.4 Systematic computations towards the FEM solution	4
3 Results and Discussion	7
3.1 Grid dependency test	7
3.2 FEM solution	8
3.3 Slope of u^n at (2,0) and (1,0)	8
4 MATLAB Script	9

List of Figures

1.1	Illustration of the domain for the boundary value problem.	1
2.1	Mesh generated by 'delaunayTriangulation' in MATLAB for $ e_0 = 0.046\%$	5
2.2	Master triangular element.	5
3.1	Change of error norm with an increasing number of nodes.	8
3.2	Nodal solution for the case with $ e_0 = 0.046\%$	9

1 PROBLEM STATEMENT

In this project, our objective is to find an approximate solution by the finite element method (FEM) of the boundary value problem (BVP) defined as follows

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = -10^3 \quad \text{in } \Omega, \quad (1.1)$$

$$u = 0 \quad \text{on } \partial\Omega. \quad (1.2)$$

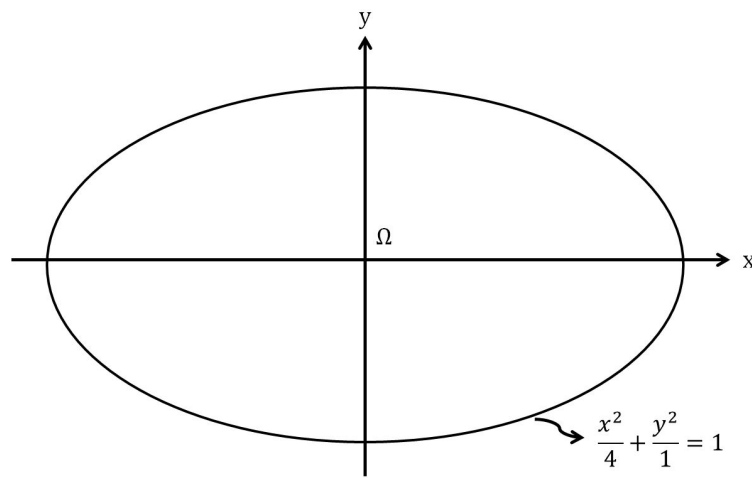


Figure 1.1: Illustration of the domain for the boundary value problem.

Figure 1.1 illustrates the domain considered for the problem which is specified as Ω . Here Ω is an elliptically shaped domain bounded by the line defined by the equation: $\frac{x^2}{4} + \frac{y^2}{1} = 1$. We will also verify the solution by performing an error analysis, and comparing the slope on boundary points found by FEM against the analytical solution.

We note that the given boundary value problem involves a partial differential equation and the highest order derivative appearing in it is 2 (i.e. $2m = 2$). By definition, in this case, boundary conditions involving derivatives of order and lower are essential and others are natural. Therefore, the only boundary condition defined by equation 1.2 of the stated problem is essential, as it only involves the derivative of order 0 (i.e. $m - 1 = 0$). Now, equation 1.1 can be written as

$$\begin{aligned} \Delta u &= -10^3, \\ \text{or } \nabla^2 u &= -10^3, \\ \text{or } \nabla \cdot \nabla u &= -10^3. \end{aligned} \quad (1.3)$$

Here ∇ is the gradient operator ($\nabla = \hat{i} \frac{\partial}{\partial x} + \hat{j} \frac{\partial}{\partial y}$ in rectangular Cartesian coordinates where $\hat{i}$ and $\hat{j}$ are unit vectors along x- and y-axes, respectively). Also, $\Delta \equiv \nabla^2$.

2 METHODOLOGY

2.1 Weak formulation

To obtain weak form of the problem, let us $\phi : \overline{\Omega} \rightarrow \mathbb{R}$ be a smooth weight function. Multiplying both sides of equation 1.3 by ϕ and integrating the result over the domain Ω we obtain

$$\int_{\Omega} \phi \nabla \cdot \nabla u d\Omega + \int_{\Omega} 10^3 \phi d\Omega = 0. \quad (2.1)$$

By expanding the first term of equation 2.1 using the chain rule of differentiation, we get

$$\int_{\Omega} \nabla \cdot (\phi \nabla u) d\Omega - \int_{\Omega} \nabla \phi \cdot \nabla u d\Omega + \int_{\Omega} 10^3 \phi d\Omega = 0. \quad (2.2)$$

Using the divergence theorem on the first term in 2.2, we get

$$\int_{\Omega} \nabla \cdot (\phi \nabla u) d\Omega = \int_{d\Omega} \phi \nabla u \cdot \hat{\mathbf{n}} d\Gamma = \int_{d\Omega} \phi \frac{\partial u}{\partial n} d\Gamma = \int_{d\Omega} \phi q_s d\Gamma. \quad (2.3)$$

Here Γ is the boundary of the domain Ω and $\hat{\mathbf{n}}$ is a unit outward normal to the boundary, $\partial\Omega$, of Ω . It is to be noted that we have set $\frac{\partial u}{\partial n} = q_s$ on $d\Omega$, which is not known. But $u = 0$ is given here as an essential condition as stated in equation 1.2. We plug the expression of equation 2.3 into equation 2.2 and get

$$\int_{d\Omega} \phi q_s d\Gamma - \int_{\Omega} \nabla \phi \cdot \nabla u d\Omega + \int_{\Omega} 10^3 \phi d\Omega = 0. \quad (2.4)$$

Now let us isolate the bilinear and linear terms of equation 2.4 and define as B and l respectively. So,

$$B(\phi, u) \equiv \int_{\Omega} \nabla \phi \cdot \nabla u d\Omega, \quad (2.5)$$

$$l(\phi) \equiv \int_{d\Omega} \phi q_s d\Gamma + \int_{\Omega} 10^3 \phi d\Omega. \quad (2.6)$$

Hence, we can write the equation 2.3 as

$$B(\phi, u) = l(\phi) \quad (2.7)$$

In order for the integrals appearing in the above definitions of B and l to be finite, $\int_{\Omega} |\nabla \phi|^2 d\Omega$, $\int_{\Omega} \phi^2 d\Omega$, and $\int_{d\Omega} \phi^2 d\Gamma$ should be finite. In fact, given the boundary $d\Omega$ where the condition is defined in the current problem, it is sufficient to require that $\int_{\Omega} |\nabla \phi|^2 d\Omega$ is finite. So, let

$$H^1 = \left\{ \psi | \psi : \overline{\Omega} \rightarrow \mathbb{R}, \int_{\Omega} \nabla \phi \cdot \nabla \phi d\Omega < \infty \right\} \quad (2.8)$$

We can now state the **weak form**, W , of the given problem as follows.

W : Find $u \in H^1$ such that $u = 0$ on $d\Omega$ and equation 2.7 holds for every $\phi \in H^1$.

2.2 Matrix formulation by Galerkin approximation

Let H^{1n} be a finite dimensional subset of H^1 and $u^n \in H^{1n}$, $\phi^n \in H^{1n}$. Then the **Galerkin approximation** of the given problem is: find $u^n \in H^{1n}$ such that $u^n = 0$ on $d\Omega$ and

$$B(\phi^n, u^n) = l(\phi^n) \quad (2.9)$$

for every $\phi^n \in H^{1n}$. Now we denote a set of basis functions $\psi_1, \psi_2, \dots, \psi_n$ in H^{1n} such that

$$\phi^n = c_i \psi_i, \quad (2.10)$$

$$u^n = d_j \psi_j. \quad (2.11)$$

where summation on a repeated index is implied and indices i and j range over 1 through n . The linearity of B and l in each of its arguments implies that

$$B(\phi^n, u^n) = B(c_i \psi_i, d_j \psi_j) = c_i B(\psi_i, \psi_j) d_j = c_i K_{ij} d_j, \quad (2.12)$$

$$l(\phi^n) = l(c_i \psi_i) = c_i l(\psi_i) = c_i F_i. \quad (2.13)$$

Here, $\mathbf{K} \equiv K_{ij}$ and $\mathbf{F} \equiv F_i$ are called the stiffness matrix and load vector, respectively, defined as

$$K_{ij} = B(\psi_i, \psi_j), \quad (2.14)$$

$$\text{and } F_i = l(\psi_i). \quad (2.15)$$

At this point equation 2.9 becomes

$$c_i K_{ij} d_j = c_i F_i \quad (2.16)$$

Since equation 2.9 holds true for every ϕ^n , therefore equation 2.16 should also hold for every choice of $c_i, c_i, \dots, c_n$, which is possible if and only if

$$K_{ij} d_j = F_i, \quad i = 1, 2, \dots, n. \quad (2.17)$$

Equation 2.17 is considered the **matrix form** of the boundary value problem. As B in our problem is a symmetric bilinear form, i.e. $B(\phi, u) = B(u, \phi)$, hence, stiffness matrix $\mathbf{K}$ is symmetric, i.e. $K_{ij} = K_{ji}$. Now let us get the original expression of B and l for our problem from 2.5 and 2.6, and then use it to get stiffness matrix $\mathbf{K}$ and load vector $\mathbf{F}$ from equation 2.14 and 2.15 respectively as follows

$$K_{ij} = \int_{\Omega} \nabla \psi_i \nabla \psi_j d\Omega, \quad (2.18)$$

$$F_i = \int_{d\Omega} \psi_i q_s d\Gamma + \int_{\Omega} 10^3 \psi_i d\Omega. \quad (2.19)$$

Consequently, when we use the K_{ij} and F_i expressions respectively from 2.18 and 2.19 and plug into equation 2.17 it comes down to the following matrix form for our specific problem.

$$\left(\int_{\Omega} \nabla \psi_i \cdot \nabla \psi_j d\Omega \right) d_j = \int_{d\Omega} \psi_i q_s d\Gamma + \int_{\Omega} 10^3 \psi_i d\Omega. \quad (2.20)$$

Now after using the definition of gradient, $\nabla = \hat{i} \frac{\partial}{\partial x} + \hat{j} \frac{\partial}{\partial y}$, we get our final form as follows

$$d_j \int_{\Omega} \left(\frac{\partial \psi_i}{\partial x} \frac{\partial \psi_j}{\partial x} + \frac{\partial \psi_i}{\partial y} \frac{\partial \psi_j}{\partial y} \right) d\Omega = \int_{d\Omega} \psi_i q_s d\Gamma + \int_{\Omega} 10^3 \psi_i d\Omega. \quad (2.21)$$

This is to be noted that to compute stiffness matrix, $\mathbf{K}$ and load vector $\mathbf{F}$, we do not need to use the term $\int_{d\Omega} \psi_i q_s d\Gamma$. Rather we enforce the essential boundary condition in the end.

At this point, we are ready for moving to the meshing step.

2.3 Meshing

We used two-dimensional triangular elements for the meshing each having three nodes; one at each vertex. We selected the necessary nodes inside the elliptic domain and performed triangulation using Delaunay triangulation (also known as Delone triangulation). We used MATLAB's 'delaunay-Triangulation' function for this meshing task. We iteratively kept increasing the number of nodes for triangulation until the following criterion was met

$$||e_0|| = \frac{||u^a - u^n||_0}{||u^a||_0} < 0.05, \quad (2.22)$$

$$\text{where } ||u^a||_0^2 = \int_{\Omega} u^{a^2} d\Omega. \quad (2.23)$$

Here, u^n is the numerical solution by the FEM, and u^a is the analytical solution which is expressed as

$$u^a = \left(\frac{2}{5} \right) \left(10^3 \right) \left(1 - \frac{x^2}{4} - \frac{y^2}{1} \right). \quad (2.24)$$

Mesh for the case with $||e_0|| = 0.046\%$ is illustrated in figure 2.1.

2.4 Systematic computations towards the FEM solution

We used the simplest master triangular element of unit base and height as illustrated in figure 2.2. Area coordinates (r, s, t) of a point P are defined as:

$$r = \frac{\text{area } P23}{\text{area } 123}; \quad s = \frac{\text{area } P31}{\text{area } 123}; \quad t = \frac{\text{area } P12}{\text{area } 123} \quad (2.25)$$

Also, r, s , and t are interrelated by the equation: $r + s + t = 1$. Now for the 3 noded triangular element

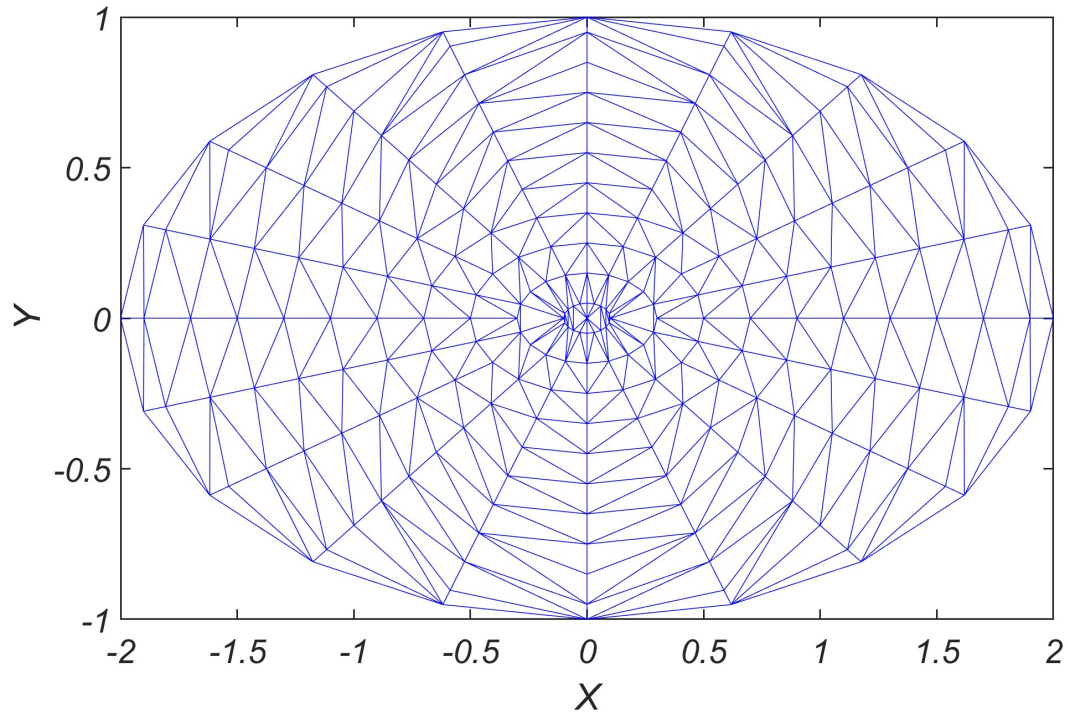


Figure 2.1: Mesh generated by 'delaunayTriangulation' in MATLAB for $\|e_0\| = 0.046\%$.

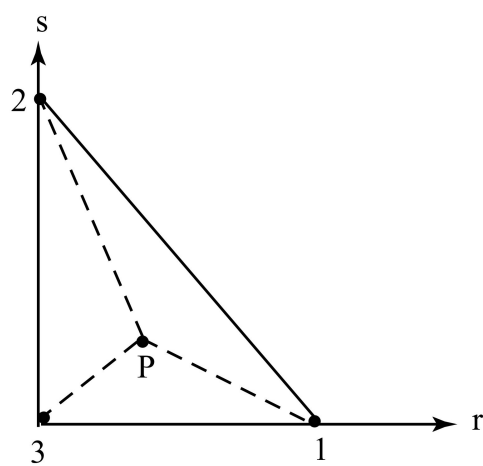


Figure 2.2: Master triangular element.

the shape functions for nodes 1, 2, and 3 are given by

$$N_1 = r, \quad N_2 = s, \quad \text{and} \quad N_3 = t. \quad (2.26)$$

For each element with 3 nodes specified as i , j , and k , the equation 2.11 becomes

$$u^e(x, y) = u_i N_1(x, y) + u_j N_2(x, y) + u_k N_3(x, y). \quad (2.27)$$

We also use the isoparametric transformation T_e that maps the master triangular element onto an actual element. Following the shape functions defined in equation 2.26, we come up with the following form of T_e for 3-noded element

$$x = x_i N_1 + x_j N_2 + x_k N_3 = x_i r + x_j s + x_k t = x_k + (x_i - x_k)r + (x_j - x_k)s, \quad (2.28)$$

$$y = y_i N_1 + y_j N_2 + y_k N_3 = y_i r + y_j s + y_k t = y_k + (y_i - y_k)r + (y_j - y_k)s. \quad (2.29)$$

Here x and y are x- and y-coordinate information of associated nodes. It is to be noted that T_e is uniquely defined for each element, and is 1-1. A necessary and sufficient condition for T_e to have a differentiable inverse is that the Jacobian, J of the transformation T_e must be non-zero as follows

$$J \equiv \det \begin{bmatrix} \frac{\partial x}{\partial r} & \frac{\partial x}{\partial s} \\ \frac{\partial y}{\partial r} & \frac{\partial y}{\partial s} \end{bmatrix} \neq 0. \quad (2.30)$$

If this condition is met, T_e^{-1} exists, it is differentiable, and it can be expressed as follows

$$r = x(y_j - y_k) - y(x_j - x_k) + [x_k(y_k - y_j) + y_k(x_j - x_k)] / A, \quad (2.31)$$

$$s = x(y_k - y_i) - y(x_k - x_i) + [x_k(y_i - y_k) + y_k(x_k - x_i)] / A, \quad (2.32)$$

$$A = (x_i - x_k)(y_j - y_k) - (y_i - y_k)(x_j - x_k). \quad (2.33)$$

For the computation of slopes of solution we need to define $\frac{\partial r}{\partial x}$, $\frac{\partial r}{\partial y}$, $\frac{\partial s}{\partial x}$, and $\frac{\partial s}{\partial y}$. For that, we note

$$dr = \frac{\partial r}{\partial x} dx + \frac{\partial r}{\partial y} dy, \quad (2.34)$$

$$ds = \frac{\partial s}{\partial x} dx + \frac{\partial s}{\partial y} dy. \quad (2.35)$$

We use this and combine with the definition of Jacobian from equation 2.30 to get

$$\frac{\partial r}{\partial x} = \frac{1}{J} \frac{\partial y}{\partial s}, \quad \frac{\partial r}{\partial y} = -\frac{1}{J} \frac{\partial x}{\partial s}, \quad \frac{\partial s}{\partial x} = -\frac{1}{J} \frac{\partial y}{\partial r}, \quad \frac{\partial s}{\partial y} = \frac{1}{J} \frac{\partial x}{\partial r}. \quad (2.36)$$

Using all this information along with equation 2.21 now we can compute the element stiffness

matrix K_{ij}^e as follows

$$K_{ij}^e = \int_{\Omega_e} \left(\frac{\partial N_i}{\partial x} \frac{\partial N_j}{\partial x} + \frac{\partial N_i}{\partial y} \frac{\partial N_j}{\partial y} \right) dx dy \quad (2.37)$$

Similarly, the element load vector F_i^e can be written as

$$F_i^e = 10^3 \int_{\Omega_e} N_i dx dy \quad (2.38)$$

Combining all the element stiffness matrices and element load vectors, we compute the global stiffness matrix, K_{ij} and global load vector F_i as follows

$$K_{ij} = \sum_e K_{ij}^e, \quad F_i = \sum_e F_i^e. \quad (2.39)$$

Now we modify the stiffness matrix and load vector to accommodate the essential boundary condition. Then we plug both the modified K_{ij} and F_i into the matrix form of the problem stated in equation 2.17. Then we invert the modified K_{ij} and multiply with F_i on the right-hand side to compute the nodal solution d_j . Finally, we obtain the complete FEM solution in the local r-s coordinate space for each element as follows

$$u_e = d_1 N_1(r, s) + d_2 N_2(r, s) + d_3 N_3(r, s). \quad (2.40)$$

3 RESULTS AND DISCUSSION

3.1 Grid dependency test

We conducted 4 simulations gradually refining the mesh (i.e increasing the number of elements and nodes). The refinement criterion is described in equation 2.22, where $\|e_0\| < 5\%$ is established as the lower bound of the required resolution. Table 3.1 lists the number of nodes and elements of the 4 simulations with associated $\|e_0\|$. The trend is illustrated in figure 3.1. Once we got to case 4, we met the criterion and proceeded with the analysis of the solution considering it our benchmark case.

Case	Nodes	Elements	$\ e_0\ $
1	89	168	0.250
2	133	252	0.087
3	177	336	0.092
4	221	420	0.046

Table 3.1: Error norm e_0 with different number of nodes and elements.

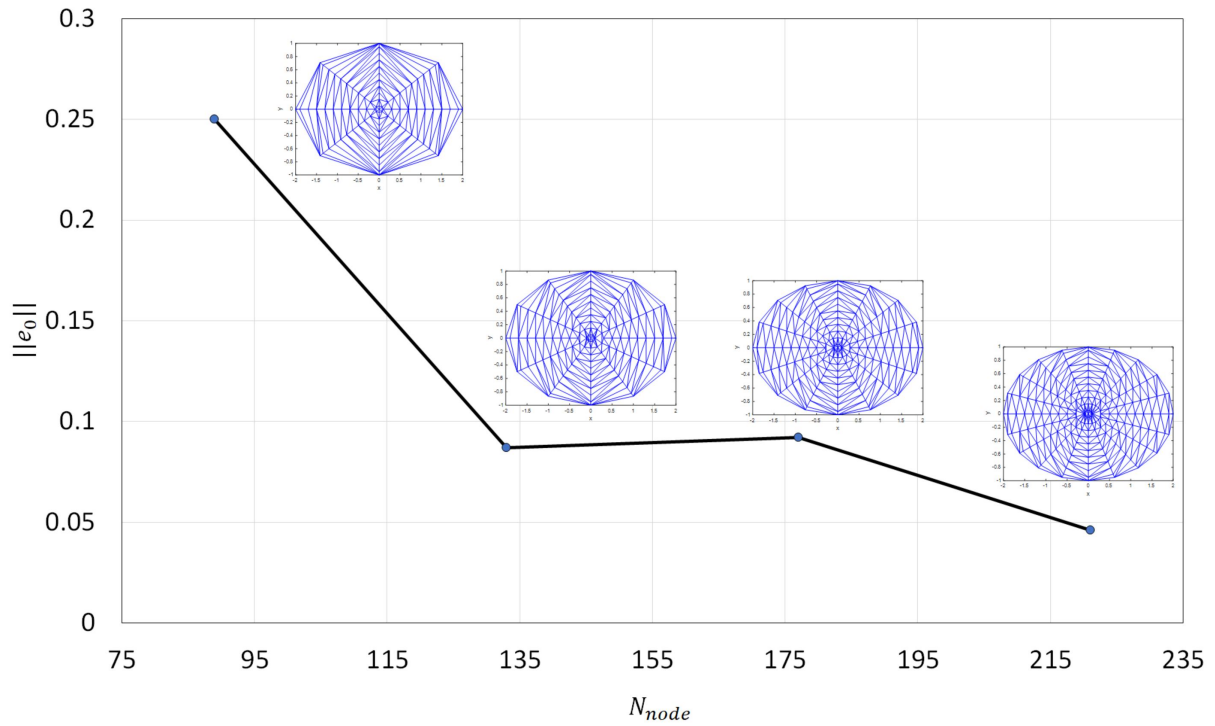


Figure 3.1: Change of error norm with an increasing number of nodes.

3.2 FEM solution

The nodal solution of the problem with 221 nodes and 420 elements is illustrated in figure 3.2. The calculated 0th order error norm, $\|e_0\|$ was calculated to be 4.6% which falls under the preset refinement criterion. It took 116 seconds of clocktime to obtain the solution.

3.3 Slope of u^n at (2,0) and (1,0)

We programmed to find all the elements that share the points (2,0) and (0,1) in the mesh and accumulated the gradients. Then we averaged them and listed them in table 3.2.

Solution	$\frac{\partial u}{\partial x}(2,0)$	$\frac{\partial u}{\partial y}(0,1)$	u_{max}
FEM	-350.85	-707.39	397.91
Analytical	-400	-800	400
% Error	12.3%	11.6%	0.52%

Table 3.2: Comparison of $\frac{\partial u}{\partial x}$, $\frac{\partial u}{\partial y}$, and u_{max} between analytical and FEM solution.

We can see that the $\frac{\partial u}{\partial x}(2,0)$ and $\frac{\partial u}{\partial y}(0,1)$ values are erroneous by about 12% which can be improved by either refining the mesh more or choosing more elements near the boundary.

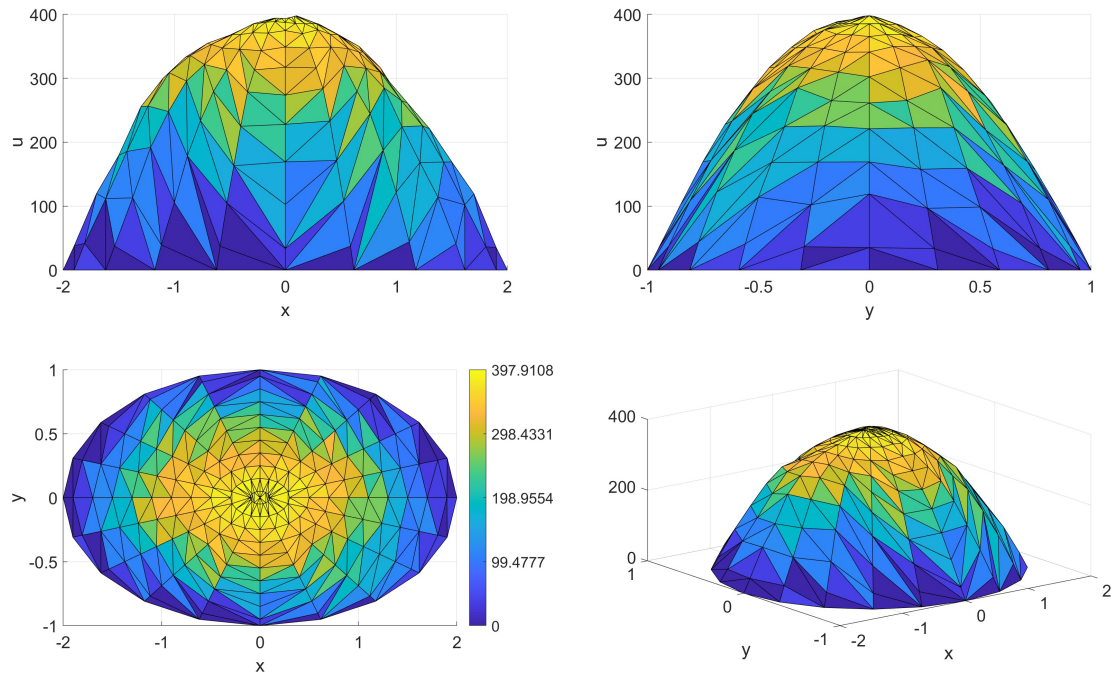


Figure 3.2: Nodal solution for the case with $||e_0|| = 0.046\%$.

4 MATLAB SCRIPT

```

1  clc
2  clear all
3
4  %% Meshing
5  a=2; % semi-major axis of ellipse
6  b=1; % semi-minor axis of ellipse
7
8  dt = pi/10; % change iteratively to refine boundary
9  t=-pi:dt:pi-dt;
10
11 x_boundary=a*cos(t); % coordinates of nodes on the boundary
12 y_boundary=b*sin(t);
13
14 nBoundary_points = length(t); % number of boundary nodes
15
16 x_interior = [];
17 y_interior = [];

```

```

18
19 dr = 0.1;
20 i = 1;
21 for f = dr/2:dr:1-dr/2
22     x_interior(i,:) = f*a*cos(t); % coordinates of nodes on the interior
23     y_interior(i,:) = f*b*sin(t);
24     i=i+1;
25 end
26
27 x_vector = [x_boundary reshape(x_interior',1,[]) 0];
28 y_vector = [y_boundary reshape(y_interior',1,[]) 0];
29
30
31 DT = delaunayTriangulation(x_vector',y_vector'); % triangular mesh
32
33 figure()
34 triplot(DT); xlabel('x'); ylabel('y');
35
36 nNodes = length(DT.Points); % total number of nodes
37 nElements = length(DT.ConnectivityList); % total number of elements
38
39 %% Finite Element Method
40 syms r s X Y
41 N = [r s 1-r-s]; % shape function in r-s domain
42
43 K_global = sym(zeros(nNodes));
44 F_global = sym(zeros(nNodes,1));
45
46 J = zeros(1,nElements);
47 xy = zeros(nElements,6);
48 dxdr = zeros(1,nElements);
49 dxds = zeros(1,nElements);
50 dydr = zeros(1,nElements);
51 dyds = zeros(1,nElements);
52
53 CM = DT.ConnectivityList; % Connectivity Matrix
54
55 for i=1:nElements

```

```
56     xy(i,:) = [DT.Points(CM(i,1),:) DT.Points(CM(i,2),:) DT.Points(CM(i,3),:)
57         ];
58     x = vpa(xy(i,1)*r + xy(i,3)*s + xy(i,5)*(1-r-s),3);
59     y = vpa(xy(i,2)*r + xy(i,4)*s + xy(i,6)*(1-r-s),3);
60
61     dxdr(i) = vpa(diff(x,r),3);
62     dxds(i) = vpa(diff(x,s),3);
63     dydr(i) = vpa(diff(y,r),3);
64     dyds(i) = vpa(diff(y,s),3);
65
66     J(i) = vpa(det([dxdr(i) dxds(i); dydr(i) dyds(i)]),3); % Jacobian
67     dNdx = vpa(diff(N,r)*dyds(i)/J(i) - diff(N,s)*dydr(i)/J(i),3);
68     dNdy = vpa(-diff(N,r)*dxds(i)/J(i) + diff(N,s)*dxdr(i)/J(i),3);
69
70     % Stiffness Matrix
71     K = vpa(int(int((dNdx'*dNdx+dNdy'*dNdy)*J(i),r,[-1 1]),s,[-1 1]),3);
72     % Force Vector
73     F = vpa(10^3*int(int(J(i)*N',r,[-1 1]),s,[-1 1]),3);
74
75     % Global Stiffness Matrix and Glob
76     for m = 1:3
77         F_global(CM(i,m),1) = F_global(CM(i,m),1) + F(m,1);
78         for n = 1:3
79             K_global(CM(i,m),CM(i,n)) = K_global(CM(i,m),CM(i,n)) + K(m,n);
80         end
81     end
82
83     %% Enforcing essential boundary condition
84     for i=1:nBoundary_points
85         K_global(i,:) = zeros;
86         K_global(i,i) = 1;
87         F_global(i,1) = 0;
88     end
89
90     %% Nodal Solution
91     u_nodes = vpa(K_global\F_global,3);
92     U = double(u_nodes);
```

```

93
94 %% Plotting nodal solution
95 figure()
96 subplot(2,2,1)
97 trisurf(CM,x_vector',y_vector',U)
98 view([0 0])
99 xlabel('x');zlabel('u')
100
101 subplot(2,2,2)
102 trisurf(CM,x_vector',y_vector',U)
103 view([90 0])
104 ylabel('y');zlabel('u')
105
106 subplot(2,2,3)
107 trisurf(CM,x_vector',y_vector',U)
108 view([0 90])
109 xlabel('x');ylabel('y')
110
111 h=colorbar;
112 t=get(h,'Limits');
113 set(h,'Ticks',linspace(t(1),t(2),5));
114
115 subplot(2,2,4)
116 trisurf(CM,x_vector',y_vector',U)
117 xlabel('x');ylabel('y')
118
119 %% Analytical Solution
120 u = 10^3*2/5*(1-X^2/4-Y^2);
121
122 %% Post Processing of nodal solution
123 num0 = zeros(1,nElements);
124 den0 = zeros(1,nElements);
125 u_FEM = sym(zeros(1,nElements));
126 dudx = sym(zeros(1,nElements));
127 dudy = sym(zeros(1,nElements));
128
129 for l = 1:nElements
130     nodes_element = CM(l,:);

```

```
131     u_FEM(1) = vpa(dot(u_nodes(nodes_element)',N),3);
132
133     x_coordinates = xy(1,1:2:end);
134     y_coordinates = xy(1,2:2:end);
135     x_in_rs = vpa(N*x_coordinates',3);
136     y_in_rs = vpa(N*y_coordinates',3);
137     u_in_rs = vpa(expand(subs(u,{X,Y},{x_in_rs,y_in_rs})),3);
138
139     n0 = vpa(expand((u_in_rs-u_FEM(1))^2),3);
140     d0 = vpa(expand((u_in_rs)^2),3);
141     num0(1) = vpa(int(int(n0*J(1),s,[0 1-r]),r,[0 1]),3);
142     den0(1) = vpa(int(int(d0*J(1),s,[0 1-r]),r,[0 1]),3);
143
144     dudx(1) = diff(u_FEM(1),r)*dyds(1)/J(1) - diff(u_FEM(1),s)*dydr(1)/J(1);
145     dudy(1) = -diff(u_FEM(1),r)*dxds(1)/J(1) + diff(u_FEM(1),s)*dxdr(1)/J(1);
146 end
147
148 %% Error
149 H0norm = vpa(sqrt(sum(num0)/sum(den0)),2)
150 %% Element number corresponding to nodes (2,0) & (0,1)
151 Ix = [];
152 h = 1;
153 for j = 1:nElements
154     for k = 1:2:width(xy)
155         if xy(j,k)==2
156             Ix(h) = j;
157             h = h+1;
158         end
159     end
160 end
161
162 Iy = [];
163 h = 1;
164 for j = 1:nElements
165     for k = 2:2:width(xy)
166         if xy(j,k)==1
167             Iy(h) = j;
168             h = h+1;
```

```
169         end
170     end
171 end
172
173 %% Gradients
174 mean(dudx(Ix))
175 std(dudx(Ix))
176
177 mean(dudy(Iy))
178 std(dudy(Iy))
```